Advanced DSA Problem-Solving Guide

12 LeetCode Problems — Approach, Topics, Algorithms & Java Code

Teaching Reference Document

oday

Contents

1 Introduction & Topic Map

This guide covers 12 classic hard/medium LeetCode problems, each mapped to a core CS topic used commonly in interviews and competitive programming. Use the table below as a quick reference when teaching — it links each topic to the problem that best demonstrates it.

#	Topic	Problem
1	Matrix Exponentiation style DP / Row-pattern DP	1411. Number of Ways to Paint N × 3 Grid
2	Combinatorics	1359. Count All Valid Pickup and Delivery Options
3	Trie + XOR (Bit Trie)	421. Maximum XOR of Two Numbers in an Array
4	SOS DP (Sum over Subsets) / Bitmask Enumeration	1178. Number of Valid Words for Each Puzzle
5	DFS + BFS / Backtracking with Memoization	301. Remove Invalid Parentheses
6	Constraint Solving / Backtracking	1307. Verbal Arithmetic Puzzle
7	Greedy Scheduling + Heap	630. Course Schedule III
8	Greedy Interval Covering	1024. Video Stitching
9	Streaming Design (TreeMap)	352. Data Stream as Disjoint Intervals
10	System Design Data Structure (O(1) Structure)	432. All O‘one Data Structure
11	Search Systems (Trie + Ranking)	642. Design Search Autocomplete System
12	Synchronization (Multithreading)	1116. Print Zero Even Odd

How to use this document for teaching: Each problem section follows the same five-part structure — *Problem Statement*, *Approach & Intuition*, *Algorithm Steps*, *Java Code*, and *Complexity + Key Teaching Points*. Encourage students to first attempt to derive the “Algorithm Steps” section themselves before revealing the code.

2 1. LeetCode 1411 — Number of Ways to Paint N × 3 Grid

Topics: Dynamic Programming, Pattern/State Compression, Combinatorics **Difficulty:** Hard

2.1 Problem Statement

You are given a grid with n rows and exactly 3 columns. Paint every cell with one of 3 colors (red, yellow, green) such that no two adjacent cells (sharing an edge, horizontally or vertically) share the same color. Return the number of ways to paint the grid, modulo 1e9 + 7.

2.2 Approach & Intuition

The key realization is that we never need to look at the *whole* grid at once — the constraint is entirely **local**: a cell only conflicts with its left/right neighbor in the same row and the cell directly above it. This means we can build the grid **row by row**, and the only information we need to carry forward from the previous row is "which pattern of colors did it use," not the colors themselves.

For a single row of 3 cells, valid colorings (no two horizontally-adjacent cells equal) fall into exactly two **pattern types**: - **Type "ABC" (all three cells different colors)** — e.g., Red-Yellow-Green. There are 3! = 6 such patterns. - **Type "ABA" (first and third cell share a color, middle is different)** — e.g., Red-Yellow-Red. There are 3 × 2 = 6 such patterns.

So every row has **12** valid colorings total, split 6/6 between the two types.

Now we ask: given the type of the previous row, how many valid next-row patterns exist (considering the vertical adjacency constraint)? - From an **ABC** row → **3** ways to form another ABC row below it, **2** ways to form an ABA row below it. - From an **ABA** row → **2** ways to form an ABC row below it, **2** ways to form an ABA row below it.

These transition counts can be derived by brute-force enumeration once and then reused — this is the "small case precomputation" trick that shows up constantly in DP problems with small fixed-width state spaces.

2.3 Algorithm Steps

1. Initialize `diff = 6` (count of ABC-type row colorings) and `same = 6` (count of ABA-type row colorings) for row 1.
2. For each subsequent row (from row 2 to row n):
   - `newDiff = diff * 3 + same * 2`
   - `newSame = diff * 2 + same * 2`
   - Update `diff, same` to these new values (mod 1e9+7).
3. After processing all n rows, the answer is `(diff + same) % MOD`.

2.4 Java Code

```java
class Solution {
    public int numOfWays(int n) {
        final long MOD = 1_000_000_007L;
        long diff = 6;  // ABC-type patterns (all 3 colors distinct)
        long same = 6;  // ABA-type patterns (first == third)

        for (int row = 2; row <= n; row++) {
            long newDiff = (diff * 3 + same * 2) % MOD;
```

```
            long newSame = (diff * 2 + same * 2) % MOD;
            diff = newDiff;
            same = newSame;
        }
        return (int) ((diff + same) % MOD);
    }
}
```

2.5 Complexity

- **Time:** O(n) — one pass over rows, O(1) work per row.
- **Space:** O(1) — only two running totals are kept.

2.6 Key Teaching Points

- Show students how to **compress row states into pattern classes** instead of tracking all $3^3 = 27$ raw colorings — this is the essence of state-space reduction in DP.
- Emphasize deriving transition coefficients (3, 2, 2, 2) by hand on a whiteboard with a 3-cell example — students remember derived formulas far better than memorized ones.
- This problem is a great segue into **matrix exponentiation**: since the transition is linear (`[diff, same]` → `[diff, same]` via a fixed 2×2 matrix), for very large n you could compute the answer in O(log n) using matrix power instead of O(n).

3 2. LeetCode 1359 — Count All Valid Pickup and Delivery Options

Topics: Combinatorics, Dynamic Programming (1-D recurrence) **Difficulty:** Hard

3.1 Problem Statement

Given `n` orders, each with one pickup (`P`) and one delivery (`D`), count the number of valid sequences of 2n events such that for every order, its pickup always occurs before its delivery. Return the count modulo `1e9 + 7`.

3.2 Approach & Intuition

Think incrementally: suppose you already have a valid arrangement for `i - 1` orders — that's a sequence of `2(i-1)` slots. To insert the `i`-th order's Pickup (P) and Delivery (D): - There are `2i - 1` total slots in the new sequence of length `2i` (i.e., `2(i-1)+1` possible positions to place P first among the existing `2(i-1)` elements). - Once P is placed, D must go **after** P, so among the remaining `2i - 1` positions, D has `i` valid choices (it must land in one of the slots after wherever P landed, out of the `2i` total slots minus P's position gives exactly `i` valid positions for D due to symmetry of the derivation).

This gives the clean recurrence:

`dp[i] = dp[i-1] * (2i - 1) * i`

with `dp[0] = 1`. This is essentially a closed-form combinatorial identity: `dp[n] = (2n)! / 2^n`.

3.3 Algorithm Steps

1. Set `result = 1`.
2. For `i` from 1 to `n`: `result = result * (2i - 1) * i mod (1e9+7)`.
3. Return `result`.

3.4 Java Code

```java
class Solution {
    public int countOrders(int n) {
        final long MOD = 1_000_000_007L;
        long result = 1;
        for (int i = 1; i <= n; i++) {
            result = (result * (2L * i - 1)) % MOD;
            result = (result * i) % MOD;
        }
        return (int) result;
    }
}
```

3.5 Complexity

- **Time:** O(n)
- **Space:** O(1)

3.6 Key Teaching Points

- Show the equivalence to the formula `(2n)! / 2^n` — dividing by 2 for each order because pickup/delivery order is fixed relative to each other out of the `2!` orderings.
- Great problem to teach **"build the recurrence by adding one unit at a time"** — a pattern that also applies to permutation-counting and probability problems.
- Compare with a naive combinatorial derivation using `C(2i-1, 2) ...` style formulas to show multiple paths to the same recurrence.

4 3. LeetCode 421 — Maximum XOR of Two Numbers in an Array

Topics: Trie (Binary/Bit Trie), Bit Manipulation, Greedy **Difficulty:** Medium

4.1 Problem Statement

Given an integer array `nums`, return the maximum result of `nums[i] XOR nums[j]` for any `i`, `j` ($0 \le i, j < n$).

4.2 Approach & Intuition

XOR is maximized when, at each bit position (starting from the most significant bit), the two numbers **disagree**. A **Bit Trie** (a trie where each node has exactly 2 children — for bit `0` and bit 1) lets us represent every number's binary form as a root-to-leaf path.

The algorithm: 1. Insert every number into the trie bit by bit, from the most significant bit (MSB) down to bit 0. 2. For every number, walk the trie again — at each bit, **greedily try to go to the opposite bit** of the current number's bit (since that maximizes the XOR contribution at that position). If the opposite branch doesn't exist, fall back to the same-bit branch. 3. Track the best XOR value found across all numbers.

This is a classic greedy-on-trie technique: locally maximizing each bit position (from MSB to LSB) guarantees a globally maximum XOR, because higher bits dominate the numeric value.

4.3 Algorithm Steps

1. Find `maxBit` = position of the highest set bit across all numbers (so we know how many bits to consider).
2. Build the trie: for each number, insert its bits from `maxBit` down to `0`.
3. For each number, traverse the trie trying to pick the **opposite bit** at every level; accumulate the resulting XOR.
4. Return the maximum XOR value seen.

4.4 Java Code

```java
class Solution {
    class TrieNode {
        TrieNode[] children = new TrieNode[2];
    }

    public int findMaximumXOR(int[] nums) {
        TrieNode root = new TrieNode();
        int maxNum = 0;
        for (int num : nums) maxNum = Math.max(maxNum, num);
        int maxBit = maxNum == 0 ? 0 : (int) (Math.log(maxNum) / Math.log(2));

        // Step 1: Build trie
        for (int num : nums) {
            TrieNode node = root;
            for (int i = maxBit; i >= 0; i--) {
                int bit = (num >> i) & 1;
```

```
                if (node.children[bit] == null) node.children[bit] = new TrieNode();
                node = node.children[bit];
            }
        }

        // Step 2: For each number, greedily find max XOR partner
        int maxXor = 0;
        for (int num : nums) {
            TrieNode node = root;
            int currXor = 0;
            for (int i = maxBit; i >= 0; i--) {
                int bit = (num >> i) & 1;
                int toggled = 1 - bit;
                if (node.children[toggled] != null) {
                    currXor |= (1 << i);
                    node = node.children[toggled];
                } else {
                    node = node.children[bit];
                }
            }
            maxXor = Math.max(maxXor, currXor);
        }
        return maxXor;
    }
}
```

4.5 Complexity

- **Time:** O(n × B) where B = number of bits (≈32) — both building and querying the trie are linear in bit-length per number.
- **Space:** O(n × B) for trie nodes.

4.6 Key Teaching Points

- Contrast this trie approach with the **hash-set prefix approach** (build candidate answer bit by bit, check via a set of number-prefixes) — both are O(n·B), but the trie generalizes better to problems with "find best XOR partner under constraint" variants (e.g., LC 1707).
- Emphasize the **greedy-per-bit-from-MSB** principle: this idea reappears in many bit-manipulation problems.
- Good problem to introduce the concept of a **Bit Trie / Binary Trie**, distinct from character tries.

5 4. LeetCode 1178 — Number of Valid Words for Each Puzzle

Topics: Bitmask, SOS DP (Sum Over Subsets) / Subset Enumeration, HashMap **Difficulty:** Hard

5.1 Problem Statement

Given a list of `words` and a list of `puzzles`, a word is valid for a puzzle if: (1) the word contains the puzzle's first letter, and (2) every letter in the word appears in the puzzle. Return, for each puzzle, the count of valid words.

5.2 Approach & Intuition

Since each word/puzzle only cares about **which distinct letters are present** (not counts or order), represent every word as a 26-bit **bitmask** (bit `i` set if letter `i` appears). Group words by bitmask in a hash map with frequency counts — this collapses potentially thousands of words into far fewer distinct masks.

For a puzzle, a word's mask is valid if: - `wordMask` is a **subset** of `puzzleMask` (every letter in the word appears in the puzzle), and - `wordMask` contains the puzzle's required first letter bit.

So for each puzzle, we need the sum of frequencies over **all subsets of `puzzleMask`** that include the first-letter bit. Instead of iterating all 2^26 masks, we use the classic **"enumerate all subsets of a bitmask"** trick (this is the essence of SOS — Sum over Subsets — DP): with at most 7 letters per puzzle, there are only `2^7 = 128` subsets to check per puzzle, using the loop `sub = (sub - 1) & mask`.

5.3 Algorithm Steps

1. Build `wordMaskCount`: a HashMap from word-bitmask → frequency, by OR-ing set bits for each word's letters.
2. For each puzzle:
   - Compute `puzzleMask` (all its letters) and `firstBit` (its first letter's bit).
   - Enumerate every non-empty subset `sub` of `puzzleMask` using the `sub = (sub - 1) & puzzleMask` trick.
   - For each subset that contains `firstBit`, add `wordMaskCount.get(sub)` (if present) to the puzzle's answer.
3. Return the list of answers.

5.4 Java Code

```java
class Solution {
    public List<Integer> findNumOfValidWords(String[] words, String[] puzzles) {
        Map<Integer, Integer> wordMaskCount = new HashMap<>();
        for (String word : words) {
            int mask = 0;
            for (char c : word.toCharArray()) {
                mask |= (1 << (c - 'a'));
            }
            wordMaskCount.merge(mask, 1, Integer::sum);
        }
```

```
        List<Integer> result = new ArrayList<>();
        for (String puzzle : puzzles) {
            int firstBit = 1 << (puzzle.charAt(0) - 'a');
            int fullMask = 0;
            for (char c : puzzle.toCharArray()) fullMask |= (1 << (c - 'a'));

            int count = 0;
            // Enumerate all subsets of fullMask (SOS subset-enumeration trick)
            int sub = fullMask;
            while (sub > 0) {
                if ((sub & firstBit) != 0) {
                    count += wordMaskCount.getOrDefault(sub, 0);
                }
                sub = (sub - 1) & fullMask;
            }
            result.add(count);
        }
        return result;
    }
}
```

5.5 Complexity

- **Time:** O(W × 26 + P × 2^7) where W = number of words, P = number of puzzles (each puzzle has exactly 7 letters, so 2^7 = 128 subsets).
- **Space:** O(W) for the mask-frequency map.

5.6 Key Teaching Points

- This is the **canonical example** to teach the subset-enumeration idiom `for (sub = mask; sub > 0; sub = (sub-1) & mask))` — make sure students trace through it on a small mask (e.g., `0b101`) on the whiteboard.
- Connect explicitly to **SOS (Sum over Subsets) DP**: this problem is SOS DP's "query per mask" brute-subset variant, useful when the mask size is small (≤ ~20) per query even if the universe is 26 bits.
- Reinforce **bitmask-as-set** representation — a recurring technique for "which distinct elements are present" problems.

6 5. LeetCode 301 — Remove Invalid Parentheses

Topics: BFS, DFS + Backtracking, String Manipulation **Difficulty:** Hard

6.1 Problem Statement

Given a string s containing lowercase letters and parentheses (and), remove the **minimum** number of invalid parentheses to make the string valid. Return all possible results (no duplicates).

6.2 Approach & Intuition

We want the **minimum number of removals**, and **all** resulting valid strings — this "minimum + all solutions" combination is a strong signal for **BFS level-order search**: BFS naturally explores states in order of "number of characters removed," so the *first* level at which we find any valid string guarantees minimality, and we simply collect every valid string found at that level.

Each BFS state is a string. From a given string, we generate all possible next states by removing exactly one parenthesis (never a letter) at each valid position. A `visited` set prevents re-exploring the same string via different removal orders (avoiding exponential blow-up from duplicate states).

An alternative equally valid approach is **DFS with pruning**: precompute how many (and) need removal by scanning left-to-right and right-to-left, then DFS-generate candidate removals while respecting those counts (a form of backtracking with memoized bounds). BFS is presented here since it directly mirrors "minimum edits" reasoning that's easy to teach.

6.3 Algorithm Steps

1. Initialize BFS queue with `s`, and a `visited` set containing `s`.
2. While the queue is not empty, process the current queue as one **level**:
   - For each string in the level: if it's valid, add to the results list and mark `found = true`.
   - If nothing valid found yet at this level, generate all single-character-removal children (only removing (or)), and enqueue unseen ones.
   - If `found` is true after processing this whole level, **stop** (do not go deeper — deeper levels would remove more characters than necessary).
3. Return the results list.

6.4 Java Code

```java
class Solution {
    public List<String> removeInvalidParentheses(String s) {
        List<String> result = new ArrayList<>();
        Set<String> visited = new HashSet<>();
        Queue<String> queue = new LinkedList<>();
        queue.offer(s);
        visited.add(s);
        boolean found = false;

        while (!queue.isEmpty()) {
            int levelSize = queue.size();
            for (int k = 0; k < levelSize; k++) {
```

```
                String curr = queue.poll();
                if (isValid(curr)) {
                    result.add(curr);
                    found = true;
                }
                if (found) continue; // don't expand further once a valid level is found

                for (int i = 0; i < curr.length(); i++) {
                    char c = curr.charAt(i);
                    if (c != '(' && c != ')') continue;
                    String next = curr.substring(0, i) + curr.substring(i + 1);
                    if (visited.add(next)) { // add() returns false if already present
                        queue.offer(next);
                    }
                }
            }
            if (found) break;
        }
        return result;
    }

    private boolean isValid(String s) {
        int balance = 0;
        for (char c : s.toCharArray()) {
            if (c == '(') balance++;
            else if (c == ')') {
                balance--;
                if (balance < 0) return false;
            }
        }
        return balance == 0;
    }
}
```

6.5 Complexity

- **Time:** O(2^n) worst case (many strings can be generated), but in practice bounded heavily by the `visited` set pruning and early stopping at the first valid level.
- **Space:** O(2^n) worst case for the visited set / queue.

6.6 Key Teaching Points

- Reinforce **"BFS finds shortest paths / minimum edits"** — draw the parallel to classic BFS-shortest-path problems even though there's no explicit graph here (the graph is implicit: nodes = strings, edges = single-character removals).
- Highlight why we must **fully finish the current level** before deciding to stop, rather than returning on the first valid string found — this guarantees we collect *all* minimal-removal solutions, not just one.
- Discuss the DFS/backtracking alternative (precompute must-remove counts of (and), then backtrack) as a good "one-up" challenge for advanced students, and compare its complexity trade-offs to BFS.

7 6. LeetCode 1307 — Verbal Arithmetic Puzzle

Topics: Constraint Solving, Backtracking, Bitmasking for used-digit tracking **Difficulty:** Hard

7.1 Problem Statement

Given an equation of the form `word1 + word2 + ... + wordN == result` where each word consists of uppercase letters, determine if there's an assignment of a distinct digit (0–9) to each distinct letter such that the equation holds numerically, with the added constraint that no leading letter of any multi-digit word can be assigned `0`.

7.2 Approach & Intuition

This is a classic **cryptarithmetic / constraint-satisfaction problem**. The naive approach (try every permutation of digits to letters) is `O(10!)` in the worst case but wasteful because it doesn't check partial validity early.

A cleaner formulation: notice that the equation `word1 + word2 + ... - result = 0` can be rewritten as a **linear equation over the letters**, where each letter c has an integer **coefficient** equal to the sum of place values (units, tens, hundreds, ...) at which it appears across all the addend words, minus its place values in the result word. For example if letter A appears in the tens place of `word1` and the units place of `result`, its coefficient is `+10 - 1 = 9`.

Once we have coefficients per letter, the problem reduces to: **assign distinct digits 0–9 to each letter such that Σ `(coefficient[c] × digit[c]) == 0`**, subject to leading letters $\neq 0$. This is solved via **backtracking**, assigning digits to letters one at a time (ideally letters with the largest `|coefficient|` first, for better pruning), while tracking used digits with a boolean array (or bitmask).

7.3 Algorithm Steps

1. Compute the coefficient of every letter by scanning each addend word (sign +1) and the result word (sign -1), accumulating place-value contributions.
2. Identify the set of "leading letters" (first character of any word with length > 1) — these can never be assigned `0`.
3. Sort letters by `|coefficient|` descending (improves pruning speed).
4. Backtrack: for each letter in order, try every unused digit 0–9 (skip 0 if it's a leading letter); maintain a running `sum`. If, after assigning all letters, `sum == 0`, a solution exists.
5. Return true/false based on whether backtracking finds a valid full assignment.

7.4 Java Code

```java
class Solution {
    public boolean isSolvable(String[] words, String result) {
        Map<Character, Integer> coeff = new HashMap<>();
        for (String w : words) addCoefficients(coeff, w, 1);
        addCoefficients(coeff, result, -1);

        Set<Character> leadingChars = new HashSet<>();
        for (String w : words) if (w.length() > 1) leadingChars.add(w.charAt(0));
        if (result.length() > 1) leadingChars.add(result.charAt(0));

        List<Character> letters = new ArrayList<>(coeff.keySet());
```

```java
        if (letters.size() > 10) return false; // more than 10 distinct letters is impossi

        // Sort by |coefficient| descending for better pruning
        letters.sort((a, b) -> Math.abs(coeff.get(b)) - Math.abs(coeff.get(a)));

        boolean[] usedDigit = new boolean[10];
        return backtrack(letters, 0, coeff, usedDigit, leadingChars, 0);
    }

    private void addCoefficients(Map<Character, Integer> coeff, String word, int sign) {
        int place = 1;
        for (int i = word.length() - 1; i >= 0; i--) {
            char c = word.charAt(i);
            coeff.put(c, coeff.getOrDefault(c, 0) + sign * place);
            place *= 10;
        }
    }

    private boolean backtrack(List<Character> letters, int idx, Map<Character, Integer> co
                              boolean[] usedDigit, Set<Character> leadingChars, int runni
        if (idx == letters.size()) {
            return runningSum == 0;
        }
        char letter = letters.get(idx);
        int coefficient = coeff.get(letter);

        for (int digit = 0; digit <= 9; digit++) {
            if (usedDigit[digit]) continue;
            if (digit == 0 && leadingChars.contains(letter)) continue;

            usedDigit[digit] = true;
            if (backtrack(letters, idx + 1, coeff, usedDigit, leadingChars,
                          runningSum + coefficient * digit)) {
                usedDigit[digit] = false;
                return true;
            }
            usedDigit[digit] = false;
        }
        return false;
    }
}
```

7.5 Complexity

- **Time:** Worst case O(10!) but with heavy pruning from the leading-zero constraint and digit-uniqueness, practically much faster — bounded by at most 10 distinct letters.
- **Space:** O(L) for the letter list and coefficient map, O(10) recursion depth.

7.6 Key Teaching Points

- Emphasize the transformation from “word arithmetic” to a **single linear equation with integer coefficients** — this reduction is the crux of the problem and a great example of turning a messy string problem into clean algebra.

- Discuss why **sorting letters by |coefficient| descending** dramatically speeds up backtracking — larger coefficients cause runningSum to diverge from 0 faster, enabling earlier pruning (this is a form of the “most constrained variable” heuristic from classic CSP theory).
- Great problem to introduce students to **Constraint Satisfaction Problems (CSPs)** formally — mention connections to Sudoku solvers and N-Queens.

8 7. LeetCode 630 — Course Schedule III

Topics: Greedy, Max-Heap (Priority Queue), Sorting **Difficulty:** Hard

8.1 Problem Statement

Given a list of courses where `courses[i] = [duration_i, lastDay_i]` (course `i` takes `duration_i` days to finish and must be finished by day `lastDay_i`), return the **maximum number of courses** you can take, taking one at a time.

8.2 Approach & Intuition

A classic **greedy + max-heap** pattern. Sort courses by their deadline (`lastDay`) ascending — this ensures we always consider the most urgent courses first. Then greedily try to take every course in this order, keeping a running `time` counter (total days used so far) and a **max-heap of durations** of courses currently "taken."

Whenever adding a new course would push `time` past its deadline, we don't just reject the new course outright — instead, we check whether **removing the longest-duration course taken so far** (from the max-heap) would free up enough time to stay under the deadline. Since we're already past a deadline, swapping out the most time-expensive course (even if it was taken earlier) can only help — it frees the most time while keeping the total count of taken courses the same until this point, giving room for future courses.

This "swap out the biggest to make room" is a hallmark **greedy exchange argument**: at any point, if we must give something up, giving up the most expensive one is never worse than giving up any other.

8.3 Algorithm Steps

1. Sort `courses` by deadline (`lastDay`) ascending.
2. Initialize `time = 0` and an empty max-heap.
3. For each course `(duration, deadline)`:
   - Add `duration` to `time`; push `duration` onto the max-heap.
   - If `time > deadline`: pop the largest duration from the heap and subtract it from `time` (i.e., "un-take" the most expensive course taken so far).
4. Return the size of the heap (number of courses successfully retained).

8.4 Java Code

```java
class Solution {
    public int scheduleCourse(int[][] courses) {
        Arrays.sort(courses, (a, b) -> a[1] - b[1]); // sort by deadline ascending
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
        int time = 0;

        for (int[] course : courses) {
            int duration = course[0], deadline = course[1];
            time += duration;
            maxHeap.offer(duration);

            if (time > deadline) {
                time -= maxHeap.poll(); // remove the longest course taken so far
```

```
            }
        }
        return maxHeap.size();
    }
}
```

8.5 Complexity

- **Time:** O(n log n) — sorting is O(n log n), and each course does at most one heap push/pop, O(log n) each.
- **Space:** O(n) for the heap.

8.6 Key Teaching Points

- This is a **must-teach example of the "greedy exchange with a heap" pattern** — the same pattern shows up in job scheduling, task deadline problems, and Huffman-style problems.
- Ask students: *"Why is it always safe to remove the longest course, rather than some other one, when we're over the deadline?"* — walking through the exchange argument builds strong greedy-algorithm intuition.
- Contrast with **DP-based** course scheduling variants (small n) to show when greedy is valid (this problem) vs. when DP/backtracking is required (general subset-sum-like constraints).

9 8. LeetCode 1024 — Video Stitching

Topics: Greedy, Interval Covering **Difficulty:** Medium

9.1 Problem Statement

Given a set of video clips `clips[i] = [start_i, end_i]`, and a target duration `time`, return the **minimum number of clips** needed so that clips fully cover the interval `[0, time]`. Return -1 if it's not possible.

9.2 Approach & Intuition

This is the canonical **greedy interval-covering / "jump game" style** problem. The key insight: at every point `i` along `[0, time)`, we only care about the **farthest reachable point** achievable by any clip that starts at or before `i`. We don't need to track *which* clip achieves that reach — just the maximum reach.

Build an array `maxReach[i]` = the farthest end among all clips starting exactly at `i`. Then simulate walking from `0` to `time - 1`, maintaining: - `currentEnd`: the farthest point covered by clips selected so far. - `farthest`: the farthest point reachable considering all clips seen up to index `i`.

Whenever we reach `i == currentEnd` (we've exhausted the reach of our currently selected set of clips), we must "use" one more clip — we jump `currentEnd` to `farthest` and increment the clip count. If `farthest` never moves past `currentEnd`, we're stuck (return -1).

This mirrors the greedy strategy in **Jump Game II**.

9.3 Algorithm Steps

1. Build `maxReach[]` of size `time`, where `maxReach[start] = max(end)` over all clips beginning at `start` (ignore clips starting ≥ time).
2. Initialize `count = 0`, `currentEnd = 0`, `farthest = 0`.
3. For `i` from `0` to `time - 1`:
   - `farthest = max(farthest, maxReach[i])`.
   - If `i == currentEnd`:
     - If `farthest <= i`: return -1 (stuck, can't extend coverage).
     - `count++`; `currentEnd = farthest`.
     - If `currentEnd >= time`: break early (fully covered).
4. Return `count` if `currentEnd >= time`, else -1.

9.4 Java Code

```java
class Solution {
    public int videoStitching(int[][] clips, int time) {
        int[] maxReach = new int[time];
        for (int[] clip : clips) {
            if (clip[0] < time) {
                maxReach[clip[0]] = Math.max(maxReach[clip[0]], clip[1]);
            }
        }

        int count = 0, currentEnd = 0, farthest = 0;
```

```
        for (int i = 0; i < time; i++) {
            farthest = Math.max(farthest, maxReach[i]);
            if (i == currentEnd) {
                if (farthest <= i) return -1;
                count++;
                currentEnd = farthest;
                if (currentEnd >= time) break;
            }
        }
        return currentEnd >= time ? count : -1;
    }
}
```

9.5 Complexity

- **Time:** O(n + time) where n = number of clips (building `maxReach`) plus a scan over `[0, time)`.
- **Space:** O(time) for the `maxReach` array.

9.6 Key Teaching Points

- Directly connect this to **Jump Game II** — students who've seen that problem will recognize the `currentEnd` / `farthest` two-pointer greedy pattern immediately.
- Discuss the **interval sorting alternative**: sort clips by start time and greedily extend coverage, picking the clip with max reach among all clips whose start ≤ current coverage — same idea, different implementation, useful if `time` is very large (avoids an O(time) array).
- Highlight the failure condition (`farthest <= i`) as the general "gap in coverage" check that appears in all interval-covering greedy problems.

10 9. LeetCode 352 — Data Stream as Disjoint Intervals

Topics: Streaming Design, TreeMap (Ordered Map), Interval Merging **Difficulty:** Hard

10.1 Problem Statement

Design a class that receives integers from a data stream one at a time (`addNum(val)`) and can return the disjoint intervals covering all integers seen so far (`getIntervals()`), sorted by interval start.

10.2 Approach & Intuition

Since numbers arrive one at a time and we must always maintain a *sorted, merged* view of intervals, a **TreeMap** (Java's balanced BST-backed ordered map) is ideal: store intervals as `start -> end` entries, keyed by `start`, giving O(log n) access to floor/ceiling neighbors.

When a new value `val` arrives: - If it's already covered by an existing interval, do nothing. - Otherwise, check its **floor neighbor** (interval with largest start ≤ val) — does its end value equal `val - 1`? If so, `val` extends that interval on the right. - Check its **ceiling neighbor** (interval with smallest start ≥ val) — does its `start` equal `val + 1`? If so, `val` extends that interval on the left. - Four cases follow: merge both neighbors together (val bridges a gap between two intervals), extend only the left neighbor, extend only the right neighbor, or insert a brand-new singleton interval `[val, val]`.

10.3 Algorithm Steps

1. Maintain `TreeMap<Integer, Integer> map` representing `start -> end` for each disjoint interval.
2. On `addNum(val)`:
   - If `map.containsKey(val)`, return early (already recorded as a start; means duplicate).
   - Get `floorEntry(val)` and `ceilingEntry(val)`.
   - Determine `mergeLeft` (floor's end == val - 1) and `mergeRight` (ceiling's start == val + 1).
   - Apply the appropriate one of 4 merge cases (both / left only / right only / neither).
3. On `getIntervals()`: iterate the TreeMap in order and build the `int[][]` result (TreeMap iteration is already sorted by key).

10.4 Java Code

```java
class SummaryRanges {
    private TreeMap<Integer, Integer> map; // start -> end

    public SummaryRanges() {
        map = new TreeMap<>();
    }

    public void addNum(int val) {
        if (map.containsKey(val)) return;

        Map.Entry<Integer, Integer> floor = map.floorEntry(val);
        Map.Entry<Integer, Integer> ceiling = map.ceilingEntry(val);
```

```
        boolean mergeLeft = floor != null && floor.getValue() + 1 == val;
        boolean mergeRight = ceiling != null && ceiling.getKey() == val + 1;

        if (mergeLeft && mergeRight) {
            map.put(floor.getKey(), ceiling.getValue());
            map.remove(ceiling.getKey());
        } else if (mergeLeft) {
            map.put(floor.getKey(), val);
        } else if (mergeRight) {
            map.remove(ceiling.getKey());
            map.put(val, ceiling.getValue());
        } else {
            map.put(val, val);
        }
    }

    public int[][] getIntervals() {
        int[][] result = new int[map.size()][2];
        int i = 0;
        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            result[i][0] = entry.getKey();
            result[i][1] = entry.getValue();
            i++;
        }
        return result;
    }
}
```

10.5 Complexity

- **Time:** O(log n) per `addNum` call (TreeMap floor/ceiling/insert/remove are all O(log n)); O(n) for `getIntervals()`.
- **Space:** O(n) intervals stored in the worst case.

10.6 Key Teaching Points

- Highlight **why a TreeMap (not a HashMap or a plain sorted list) is the right structure** here: we need efficient sorted-order neighbor queries (`floorEntry`/`ceilingEntry`) on every insert — a plain array would need O(n) shifting, and a HashMap has no ordering.
- Walk through **all four merge cases** on the whiteboard with a concrete example stream, e.g., `1, 3, 7, 2, 6` — this cements the "bridge the gap" mental model.
- This is a strong problem to introduce **streaming/online algorithm design** where a data structure must support incremental updates instead of one-shot batch processing.

11 10. LeetCode 432 — All O‘one Data Structure

Topics: System Design Data Structure, Doubly Linked List, HashMap **Difficulty:** Hard

11.1 Problem Statement

Design a data structure that supports, all in **O(1)** average time: - `inc(key)`: increment the count of key by 1 (insert with count 1 if not present). - `dec(key)`: decrement the count of key by 1 (remove key entirely if count becomes 0). - `getMaxKey()`: return any key with the maximum count (empty string if none). - `getMinKey()`: return any key with the minimum count (empty string if none).

11.2 Approach & Intuition

The core challenge is maintaining **O(1) access to both the min and max frequency keys** while frequencies change dynamically. A **doubly linked list of “buckets”**, each bucket representing a distinct count value and holding the *set* of keys currently at that count, solves this elegantly:

- The linked list is kept **sorted by count** at all times (ascending from head to tail).
- Two sentinel nodes (head with count = −∞ and `tail` with count = +∞) simplify edge cases.
- A `HashMap<String, Bucket>` gives O(1) lookup of which bucket a given key currently belongs to.

To `inc(key)`: find the key’s current bucket (or none), move the key to the *next* bucket in the list with `count + 1` (creating that bucket right after the current one if it doesn’t already exist with the correct count), and remove the key from its old bucket (deleting the bucket if it becomes empty). `dec(key)` is symmetric, moving to the *previous* bucket.

`getMaxKey()`/`getMinKey()` become trivial O(1) lookups: peek at any key in `tail.prev` (max) or `head.next` (min).

11.3 Algorithm Steps

1. Maintain sentinel head (count = -∞) and `tail` (count = +∞) doubly-linked Bucket nodes, plus `keyToBucket: HashMap<String, Bucket>`.
2. `inc(key)`:
   - If key unseen: ensure a bucket with count 1 exists right after head (create if needed); add key to it.
   - If key seen: let `curr` = its bucket. Ensure a bucket with count `curr.count + 1` exists right after `curr` (create if needed); move key into it; remove key from `curr`; delete `curr` if empty.
3. `dec(key)`:
   - Let `curr` = key’s bucket. If `curr.count == 1`, remove the key entirely (no bucket needed).
   - Else ensure a bucket with count `curr.count - 1` exists right before `curr` (create if needed); move key into it.
   - Remove key from `curr`; delete `curr` if empty.
4. `getMaxKey()` = any key in `tail.prev` (or “” if `tail.prev == head`).
5. `getMinKey()` = any key in `head.next` (or “” if `head.next == tail`).

11.4 Java Code

```java
class AllOne {
    class Bucket {
        int count;
        Set<String> keys = new LinkedHashSet<>();
        Bucket prev, next;
        Bucket(int count) { this.count = count; }
    }

    private final Bucket head, tail;
    private final Map<String, Bucket> keyToBucket;

    public AllOne() {
        head = new Bucket(Integer.MIN_VALUE);
        tail = new Bucket(Integer.MAX_VALUE);
        head.next = tail;
        tail.prev = head;
        keyToBucket = new HashMap<>();
    }

    private Bucket insertAfter(Bucket node, int count) {
        Bucket newBucket = new Bucket(count);
        newBucket.prev = node;
        newBucket.next = node.next;
        node.next.prev = newBucket;
        node.next = newBucket;
        return newBucket;
    }

    private void removeBucket(Bucket node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    public void inc(String key) {
        if (!keyToBucket.containsKey(key)) {
            Bucket first = head.next;
            if (first.count != 1) first = insertAfter(head, 1);
            first.keys.add(key);
            keyToBucket.put(key, first);
        } else {
            Bucket curr = keyToBucket.get(key);
            Bucket next = curr.next;
            if (next.count != curr.count + 1) next = insertAfter(curr, curr.count + 1);
            next.keys.add(key);
            keyToBucket.put(key, next);

            curr.keys.remove(key);
            if (curr.keys.isEmpty()) removeBucket(curr);
        }
    }

    public void dec(String key) {
```

```
        Bucket curr = keyToBucket.get(key);
        if (curr.count == 1) {
            keyToBucket.remove(key);
        } else {
            Bucket prev = curr.prev;
            if (prev.count != curr.count - 1) prev = insertAfter(curr.prev, curr.count - 1
            prev.keys.add(key);
            keyToBucket.put(key, prev);
        }
        curr.keys.remove(key);
        if (curr.keys.isEmpty()) removeBucket(curr);
    }

    public String getMaxKey() {
        return tail.prev == head ? "" : tail.prev.keys.iterator().next();
    }

    public String getMinKey() {
        return head.next == tail ? "" : head.next.keys.iterator().next();
    }
}
```

11.5 Complexity

- **Time:** O(1) amortized for all four operations — linked list insert/remove and hash map access are all O(1).
- **Space:** O(k) where k = number of distinct keys.

11.6 Key Teaching Points

- This problem is the **flagship example of combining a HashMap with a Doubly Linked List** to achieve true O(1) operations — the same underlying idea (buckets keyed by frequency) is reused in **LFU Cache (LC 460)**, making this a great problem to pair in a lesson.
- Emphasize *why* an ordinary max-heap/min-heap approach **fails** the O(1) requirement (heap operations are O(log n)) — this motivates the bucket-list design.
- Walk through a live example: `inc("a")`, `inc("a")`, `inc("b")`, `dec("a")` — draw the bucket list evolving on the whiteboard step by step; this visualization is the single best teaching tool for this problem.

12 11. LeetCode 642 — Design Search Autocomplete System

Topics: Search Systems, Trie, Heap/Sorting, HashMap **Difficulty:** Hard

12.1 Problem Statement

Design a search autocomplete system for a search engine. Given historical `sentences` and their `times` (frequency counts), implement `input(c)`: as the user types character by character, return the **top 3** historical sentences that start with the currently typed prefix, ranked by frequency (descending), then lexicographically (ascending) for ties. Typing # signifies the end of a sentence — record it (incrementing its frequency, or adding it fresh with frequency 1) and reset the current input.

12.2 Approach & Intuition

This is a classic **Trie + ranking** search-system design problem. Build a trie over all historical sentences, but instead of only marking "end of word" at leaf nodes, **store a map of `sentence -> frequency` at every node along each sentence's path**. This means each trie node "knows" all completions passing through it, along with their frequencies — enabling O(1)-ish lookup for "what sentences pass through this prefix" without re-walking anything.

As the user types each character, we maintain a `current` pointer that walks down the trie one node per character (rather than re-searching from the root every keystroke — this incremental walk is the performance-critical design choice). At each keystroke, we grab `current.counts` — the map of sentence → frequency at that node — and use a **priority queue (max-heap by frequency, tie-broken lexicographically)** to extract the top 3.

On #, we add the just-typed sentence to the trie (incrementing frequency if it already existed), then reset `current` to the trie root and clear the input buffer.

12.3 Algorithm Steps

1. **Initialization:** For each `(sentence, count)` pair, insert into the trie: walk/create nodes for each character, and at every node visited, add `count` to `node.counts[sentence]`.
2. **`input(c)`:**
   - If `c == '#'`: insert `currentInput` into the trie with count 1 (merging into existing frequency if present); reset `current = root`, clear `currentInput`; return empty list.
   - Else: append `c` to `currentInput`; move `current = current.children.get(c)` (or null if no such branch).
   - If `current == null`: return empty list (no sentences match this prefix).
   - Otherwise, take `current.counts.entrySet()`, push into a priority queue ordered by (frequency descending, then sentence lexicographically ascending), and pop the top 3.

12.4 Java Code

```
class AutocompleteSystem {
    class TrieNode {
        Map<Character, TrieNode> children = new HashMap<>();
        Map<String, Integer> counts = new HashMap<>(); // sentence -> frequency, stored al
    }
```

```java
    private final TrieNode root;
    private TrieNode current;
    private StringBuilder currentInput;

    public AutocompleteSystem(String[] sentences, int[] times) {
        root = new TrieNode();
        for (int i = 0; i < sentences.length; i++) {
            addSentence(sentences[i], times[i]);
        }
        current = root;
        currentInput = new StringBuilder();
    }

    private void addSentence(String sentence, int count) {
        TrieNode node = root;
        for (char c : sentence.toCharArray()) {
            node.children.putIfAbsent(c, new TrieNode());
            node = node.children.get(c);
            node.counts.merge(sentence, count, Integer::sum);
        }
    }

    public List<String> input(char c) {
        if (c == '#') {
            addSentence(currentInput.toString(), 1);
            currentInput = new StringBuilder();
            current = root;
            return new ArrayList<>();
        }

        currentInput.append(c);
        if (current != null) {
            current = current.children.get(c);
        }
        if (current == null) return new ArrayList<>();

        PriorityQueue<Map.Entry<String, Integer>> pq = new PriorityQueue<>(
            (a, b) -> a.getValue().equals(b.getValue())
                ? a.getKey().compareTo(b.getKey())
                : b.getValue() - a.getValue()
        );
        pq.addAll(current.counts.entrySet());

        List<String> result = new ArrayList<>();
        for (int i = 0; i < 3 && !pq.isEmpty(); i++) {
            result.add(pq.poll().getKey());
        }
        return result;
    }
}
```

12.5 Complexity

- **Time:** O(L) per character typed to walk the trie, plus O(M log 3) (or O(M log M)) to rank the M sentences passing through the current node using the heap.
- **Space:** O(Σ |sentence| × average prefix depth) — storing frequency maps at every trie node can be memory-heavy; a production system would cap or lazily rank instead.

12.6 Key Teaching Points

- This is an excellent **real-world system design tie-in**: discuss how real search engines (type-ahead / autocomplete) use similar trie-based ranking, but at scale would need **top-K precomputation per node** (rather than re-ranking on every keystroke) to stay fast — a nice bridge from LeetCode to real systems.
- Point out the **incremental trie-pointer walk** (`current` persists across `input()` calls) as the key optimization over naively re-searching the whole trie from the root on every keystroke.
- Discuss the trade-off of storing `counts` maps at *every* node (fast queries, higher memory) vs. only storing frequencies at leaf/end nodes and aggregating via DFS on each query (less memory, slower queries) — a good discussion question for advanced students.

13 12. LeetCode 1116 — Print Zero Even Odd

Topics: Synchronization, Multithreading, wait/notify **Difficulty:** Medium

13.1 Problem Statement

Given a class with three threads calling `zero()`, `even()`, and `odd()` respectively, and a shared `printNumber` callback, coordinate the threads so the combined output is `0`, then the first odd number, `0`, then the first even number, `0`, then the next odd number, and so on — producing sequence `0 1 0 2 0 3 0 4 ...` up to `n`.

13.2 Approach & Intuition

This is a classic **thread synchronization / producer-consumer coordination** problem, solvable with Java's intrinsic lock (`synchronized`) plus `wait()`/`notifyAll()`. The key design choice is a shared **state variable** indicating whose turn it is: - `state == 0`: it's `zero`'s turn to print `0`. - `state == 1`: it's odd's turn to print the next odd number. - `state == 2`: it's even's turn to print the next even number.

All three threads share a single lock object. Each thread runs a loop: while it's not its turn (`state` doesn't match), it calls `lock.wait()` to sleep, releasing the lock. When woken by `notifyAll()`, it re-checks the condition (always in a `while` loop, **never** an `if`, to correctly handle spurious wakeups). Once it's really its turn, the thread prints, updates `state` to hand off control to the next thread, and calls `notifyAll()` to wake all waiting threads (only the one whose new `state` matches will actually proceed).

The `zero` thread alternates handing off to odd or even depending on whether the *next* number to print is odd or even — this alternation logic lives entirely inside `zero()`, keeping `odd()` and `even()` simple.

13.3 Algorithm Steps

1. Shared fields: `int n`, `int state = 0` (0 = zero's turn, 1 = odd's turn, 2 = even's turn), and a lock object.
2. `zero()`: loop `n` times — wait until `state == 0`; print `0`; set `state` to 1 if the next number (`i`, 1-indexed) is odd, else 2; `notifyAll()`.
3. `odd()`: loop over odd numbers `1, 3, 5, ...` up to `n` — wait until `state == 1`; print the number; set `state = 0`; `notifyAll()`.
4. `even()`: loop over even numbers `2, 4, 6, ...` up to `n` — wait until `state == 2`; print the number; set `state = 0`; `notifyAll()`.

13.4 Java Code

```java
import java.util.function.IntConsumer;

class ZeroEvenOdd {
    private final int n;
    private int state = 0; // 0 = zero's turn, 1 = odd's turn, 2 = even's turn
    private final Object lock = new Object();

    public ZeroEvenOdd(int n) {
        this.n = n;
    }
```

```
    public void zero(IntConsumer printNumber) throws InterruptedException {
        synchronized (lock) {
            for (int i = 1; i <= n; i++) {
                while (state != 0) lock.wait();
                printNumber.accept(0);
                state = (i % 2 == 1) ? 1 : 2; // hand off to odd() or even()
                lock.notifyAll();
            }
        }
    }

    public void even(IntConsumer printNumber) throws InterruptedException {
        synchronized (lock) {
            for (int i = 2; i <= n; i += 2) {
                while (state != 2) lock.wait();
                printNumber.accept(i);
                state = 0;
                lock.notifyAll();
            }
        }
    }

    public void odd(IntConsumer printNumber) throws InterruptedException {
        synchronized (lock) {
            for (int i = 1; i <= n; i += 2) {
                while (state != 1) lock.wait();
                printNumber.accept(i);
                state = 0;
                lock.notifyAll();
            }
        }
    }
}
```

13.5 Complexity

- **Time:** O(n) total prints across all threads; each thread does O(n/2) or O(n) iterations.
- **Space:** O(1) beyond the shared state.

13.6 Key Teaching Points

- Stress the golden rule of Java concurrency: **always re-check the wait condition in a `while` loop, never an `if`** — this correctly handles both spurious wakeups and the case where `notifyAll()` wakes a thread whose turn hasn't actually arrived yet.
- Explain why `notifyAll()` is used instead of `notify()`: with 3 different threads potentially waiting, `notify()` might wake the *wrong* thread (one whose condition still isn't met), causing a deadlock; `notifyAll()` wakes everyone, and only the correctly-conditioned thread proceeds while others simply re-wait.
- This problem is a good entry point into discussing alternative synchronization primitives — e.g., `java.util.concurrent.Semaphore` — which many students find more intuitive than raw `wait()`/`notify()`. Consider showing a semaphore-based rewrite as an extension exercise.

14 Summary Table for Quick Revision

Problem	Core Technique	Time Complexity
1411. Paint N×3 Grid	Row-pattern DP (2-state)	O(n)
1359. Pickup and Delivery	Incremental combinatorics	O(n)
421. Max XOR of Two Numbers	Bit Trie + greedy	O(n·32)
1178. Valid Words for Puzzle	Bitmask + subset enumeration (SOS)	O(W·26 + P·2^7)
301. Remove Invalid Parentheses	BFS level-order search	O(2^n) worst case
1307. Verbal Arithmetic Puzzle	Backtracking CSP	O(10!) worst case
630. Course Schedule III	Greedy + max-heap	O(n log n)
1024. Video Stitching	Greedy interval covering	O(n + time)
352. Data Stream as Disjoint Intervals	TreeMap	O(log n) per op
432. All O‘one Data Structure	HashMap + doubly linked list buckets	O(1) amortized
642. Autocomplete System	Trie + heap ranking	O(L) per keystroke
1116. Print Zero Even Odd	wait/notify synchronization	O(n)